

Stage réalisé à la Société Tunisienne de Banque (STB)

Rapport de Stage

STB StageConnect

Plateforme Intelligente de Gestion des Stagiaires

Stagiaire :	Abdelkader KADDACHI
Maître de stage :	M. Makrem TAIEB
Organisme :	Société Tunisienne de Banque (STB)
Formation :	Ingénieur en Informatique — ESPRIT
Période :	Été 2026 — 8 semaines

.NET 8

Spring Boot

Angular 18

RabbitMQ

Docker

K8s

SharpAPI AI

STB StageConnect — Plateforme Intelligente de Gestion des Stagiaires

Stagiaire : Abdelkader KADDACHI

Formation : Diplôme National d'Ingénieur en Informatique — ESPRIT 2026

Organisme d'accueil : Société Tunisienne de Banque (STB)

Maître de stage : M. Makrem TAIEB

Période : Été 2026 (8 semaines)

Introduction Générale

Le présent rapport rend compte du stage de fin d'études réalisé au sein de la Société Tunisienne de Banque (STB) dans le cadre de l'obtention du Diplôme National d'Ingénieur en Informatique à l'ESPRIT (École Supérieure Privée d'Ingénierie et de Technologies).

Ce stage de 8 semaines a conduit à la conception, au développement et au déploiement d'une plateforme logicielle complète baptisée STB StageConnect, dédiée à la gestion numérique du cycle de vie des stagiaires au sein de la STB. La solution mobilise des technologies de pointe — architecture microservices hybride Java/.NET 8, Intelligence Artificielle, communication temps réel, génération automatique de documents PDF — et respecte les exigences de sécurité propres au secteur bancaire.

Contexte & Problématique

Contexte Organisationnel

Fondée le 3 février 1957, la Société Tunisienne de Banque (STB) est le premier établissement bancaire national créé au lendemain de l'indépendance tunisienne. Disposant d'un réseau de plus de 160 agences, elle joue un rôle moteur dans le financement de l'économie tunisienne. Chaque année, la STB accueille des dizaines de stagiaires issus des meilleures écoles d'ingénieurs et universités tunisiennes, répartis dans ses directions techniques (DSI, Monétique, Sécurité, Banque Digitale).

Problématique Identifiée

Avant ce projet, la gestion des stagiaires reposait intégralement sur des processus manuels (emails, fichiers Excel, déplacements physiques), engendrant :

- Des délais d'attente prolongés pour les étudiants et un risque élevé de perte des meilleurs profils ;
- Des inadéquations fréquentes entre les compétences réelles du stagiaire et la complexité du projet attribué, due à l'absence d'analyse structurée des CVs ;
- Une absence totale de traçabilité en temps réel du statut d'avancement des dossiers ;
- Des conventions et attestations générées manuellement, sujettes à des erreurs et des retards administratifs.

Objectifs du Projet

Le projet STB StageConnect a été initié pour apporter une réponse technologique moderne et pérenne à ces défis :

1. Digitaliser à 100% le cycle de vie du stagiaire : de l'inscription initiale jusqu'à la délivrance de l'attestation finale ;
2. Intégrer l'Intelligence Artificielle au cœur du processus RH : automatiser l'analyse sémantique des CVs (PDF/DOCX) via des modèles NLP avancés et recommander instantanément le sujet de stage optimal à l'administrateur ;
3. Garantir une expérience temps réel et collaborative : doter la plateforme d'un centre de notification unifié poussant instantanément les événements métiers vers l'ensemble des acteurs concernés ;
4. Mettre en place une architecture résiliente et sécurisée : respectant les exigences rigoureuses du secteur bancaire (microservices indépendants, chiffrement JWT, auditabilité complète, déploiement conteneurisé).

Chapitre 1 — Contextualisation & Méthodologie Agile

1.1 Présentation de l'Organisme d'Accueil : STB Bank

La STB est le premier établissement bancaire public tunisien créé après l'indépendance (1957). Le stage a été réalisé au sein de la Direction du Capital Humain, en étroite collaboration avec les équipes de développement logiciel et de transformation digitale.

1.2 Méthodologie de Travail : Scrum Adapté

Afin de maîtriser les risques techniques et d'intégrer rapidement les retours utilisateurs, le projet a été conduit selon la méthodologie Scrum articulée autour de 4 releases majeures de 2 semaines chacune :

Release	Période	Périmètre Fonctionnel & Technique
Release 1	Semaines 1-2	Socle architectural : Spring Cloud Gateway, Auth Service
Release 2	Semaines 3-4	Microservice Stagiaire (candidatures, upload CV), Conve
Release 3	Semaines 5-6	Évaluations mi-parcours/finale, attestations de stage,
Release 4	Semaines 7-8	Module IA d'appariement (SubjectMatching.Service), inté

Tableau 1.1 — Découpage temporel des releases du projet STB StageConnect

1.3 Outils de Gestion de Projet

- Contrôle de version : Git / GitHub avec branches de feature (feature/*), pull requests et revue de code ;
- Suivi des tâches : Kanban board GitHub Projects (colonnes : Backlog → In Progress → Review → Done) ;
- Documentation : README.md détaillé, commentaires XML sur les APIs, Swagger UI par microservice.

Chapitre 2 — Architecture Système & Choix Technologiques

2.1 Vue d'Ensemble de l'Architecture Hybride

Le système STB StageConnect adopte une architecture orientée microservices hybride, tirant le meilleur parti des écosystèmes Java Spring Boot et Microsoft .NET 8, organisée en trois couches :

Couche d'Infrastructure Java / Spring Cloud :

Chargée de la passerelle réactive (Spring Cloud Gateway sur port 18080), de l'authentification centrale (Auth Service sur port 8081 avec base MySQL), et de la découverte dynamique des services (Eureka sur port 8761).

Couche Métier .NET 8 :

Composée de 5 microservices indépendants spécialisés, chacun disposant de sa propre

base PostgreSQL dédiée pour une isolation complète des domaines (pattern Database-per-Service).

Couche Événementielle Asynchrone :

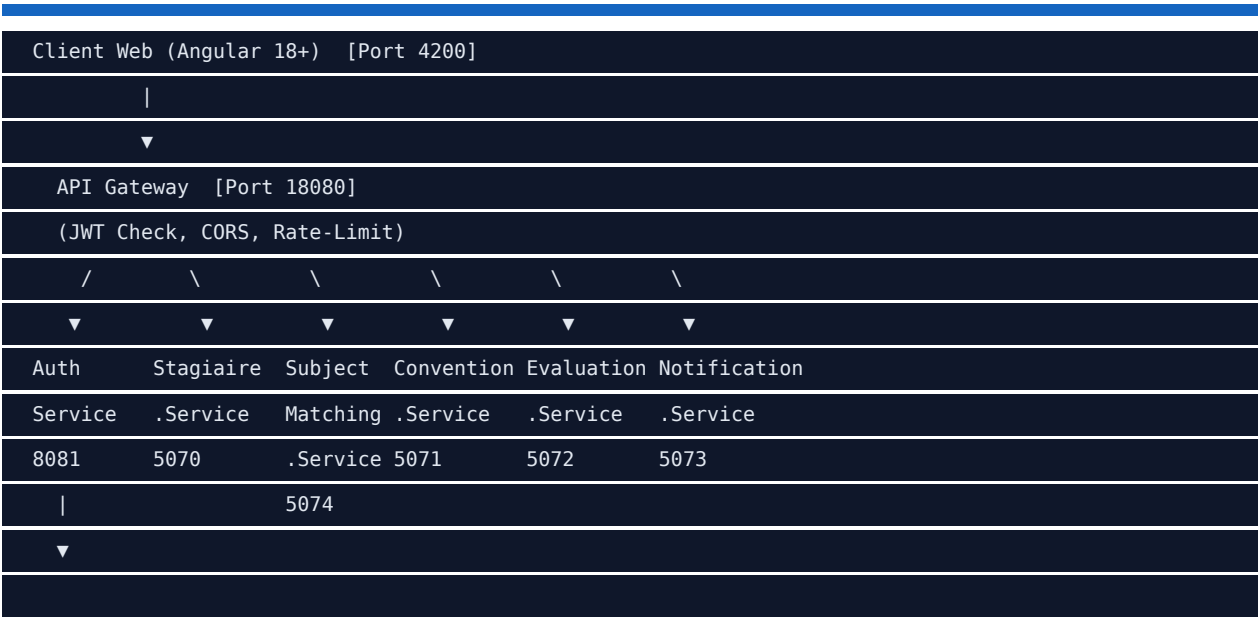
Broker RabbitMQ 3.13 exploitant le framework MassTransit pour découpler la génération des documents, les alertes IA et les envois de notifications.

2.2 Cartographie des Microservices

Composant	Technologie	Port	Base de Données	Rôle Principal
API Gateway	Spring Cloud Gateway	18080	—	Point d'entrée unique, vérification JWT
Auth Service	Spring Boot 3.2	8081	MySQL 8.0	Gestion des comptes, tokens JWT & re
Eureka Server	Spring Cloud Netflix	8761	—	Annuaire et découverte dynamique d
Config Server	Spring Cloud Config	8888	—	Distribution centralisée des configura
Stagiaire.Service	.NET 8 / EF Core	5070	PostgreSQL 15	Candidatures, stagiaires, stockage de
SubjectMatching.Serv	.NET 8 / EF Core	5074	PostgreSQL 15	Sujets de stage, analyse IA CVs (Shar
Convention.Service	.NET 8 / QuestPDF	5071	PostgreSQL 15	Génération automatique des convent
Evaluation.Service	.NET 8 / QuestPDF	5072	PostgreSQL 15	Évaluations encadrant, notes /20, att
Notification.Service	.NET 8 / SignalR	5073	PostgreSQL 15	Hub WebSocket SignalR, emails trans
Frontend Client	Angular 18+ / Tailwind	4200	—	Single Page Application réactive avec

Tableau 2.1 — Cartographie complète des microservices de la plateforme STB StageConnect

2.3 Diagramme de Flux Global



MySQL

PostgreSQL (5 bases dédiées)



2.4 Bibliothèque Partagée : Smartek.Common

Pour éviter la duplication de code entre les 5 microservices .NET, une bibliothèque commune Smartek.Common a été développée. Elle expose :

- JwtMiddleware : extraction et validation du token JWT à chaque requête ;
- MetricsExtensions (UseSmartekMetrics()) : enregistrement des métriques Prometheus-net, exposition de l'endpoint /metrics ;
- HealthCheckExtensions : endpoints /health/live (liveness) et /health/ready (readiness avec DB) ;
- GlobalExceptionHandler : gestion unifiée des erreurs avec format RFC 7807 ;
- AuditMiddleware : traçabilité automatique de toutes les actions sensibles ;
- StorageService : abstraction du stockage fichier (volumes Docker / Kubernetes PVC).

Chapitre 3 — Release 1 : Socle d'Infrastructure & Sécurité

3.1 API Gateway (Spring Cloud Gateway)

La passerelle réactive constitue le point d'entrée unique de la plateforme. Toutes les requêtes transitent par le port 18080 via les routes unifiées /api/v1/**. Aucun microservice n'est directement accessible depuis l'extérieur.

Fonctionnalités clés :

- Vérification JWT centralisée à chaque requête avant routage vers les microservices aval ;
- Découverte dynamique des instances via le registre Eureka (LoadBalancer côté gateway) ;
- CORS unifié : liste des origines autorisées configurable via variable d'environnement GATEWAY_ALLOWED_ORIGINS ;
- Rate limiting par IP pour protéger l'infrastructure contre les abus ;
-

Propagation du Traceld dans les en-têtes HTTP pour la traçabilité distribuée.

3.2 Auth Service (Spring Boot 3.2 / MySQL)

L'Auth Service centralise toute la gestion des identités et de l'authentification. Il stocke ses données dans une base MySQL 8.0 (smartek_db), distincte des bases PostgreSQL métier.

Fonctionnalités :

- Inscription, connexion, gestion des profils (Admin RH, Stagiaire, Encadrant) ;
- Authentification stateless JWT : tokens signés en HMAC-SHA256 (JWT_SECRET partagé de 48 octets) ;
- Refresh tokens rotatifs : stockés en base MySQL, renouvelés automatiquement via l'intercepteur Angular ;
- Hachage BCrypt des mots de passe (force 10) ;
- Initialisation automatique de l'administrateur au premier démarrage (seed via variables d'environnement) ;
- Création des comptes Encadrant par l'administrateur (événement EncadrantCreated → Notification.Service → email de bienvenue).

3.3 Eureka Server & Config Server

- Eureka Server (port 8761) : annuaire de découverte dynamique permettant aux services de s'enregistrer et de se localiser sans configuration statique d'adresses IP ;
- Config Server (port 8888) : distribution centralisée des configurations environnementales, permettant de modifier les paramètres sans reconstruire les images Docker.

3.4 Gestion des Rôles & Sécurité Granulaire

Trois rôles distincts avec isolation stricte des données :

Rôle	Description	Périmètre d'accès
ADMIN	Administrateur RH	Accès complet à tous les dossiers, tableaux de bord, st
TRAINER (Encadrant)	Tuteur pédagogique	Uniquement ses stagiaires affectés, évaluations, journa
LEARNER (Stagiaire)	Étudiant	Uniquement son propre dossier, convention, sujet, évalu

Chapitre 4 — Release 2 : Gestion des Candidatures & Conventions PDF

4.1 Stagiaire.Service (.NET 8 / EF Core)

Développé sur le port 5070 avec sa base de données dédiée `stagiaire_service_db`, ce microservice gère l'intégralité du dossier candidat.

Fonctionnalités principales :

- Formulaire de candidature : saisie des informations personnelles, académiques et professionnelles ;
- Upload sécurisé du CV (PDF) et de la lettre de motivation sur un volume Docker persistant (`stagiaire-storage`) ;
- Journal de bord hebdomadaire : saisie des tâches par le stagiaire, validation et commentaires par l'encadrant ;
- Journal d'audit consolidé : traçabilité exhaustive de toutes les actions administratives (horodatage, utilisateur, action) ;
- Health checks : endpoint `/health/live` (liveness process) et `/health/ready` (readiness incluant la DB).

Modèle de données clé :

```
[Table("candidatures")]
public class Candidature {
    [Key] public Guid Id { get; set; } = Guid.NewGuid();
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Department { get; set; } = string.Empty;
    public CandidatureStatus Status { get; set; } = CandidatureStatus.Pending;
    public string? CvFilePath { get; set; }
    public DateTime SubmittedAt { get; set; } = DateTime.UtcNow;
}
```

4.2 Convention.Service (.NET 8 / QuestPDF)

Développé sur le port 5071 avec sa base dédiée `convention_service_db`, ce microservice automatise la génération des conventions de stage officielles.

Fonctionnalités :

- Déclenchement automatique : à la réception de l'événement `CandidatureAccepted` via RabbitMQ ;
-

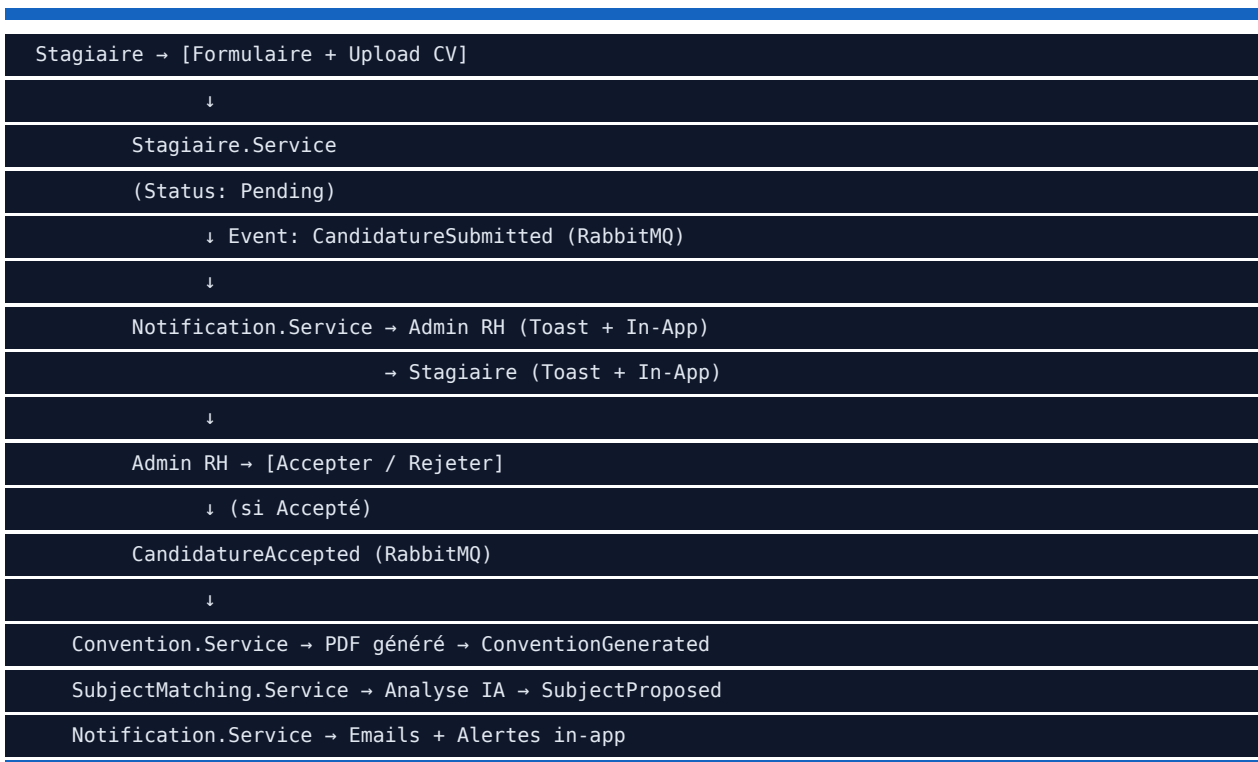
Génération PDF temps réel avec la bibliothèque QuestPDF : mise en page institutionnelle STB, données personnalisées du stagiaire, dates, département, encadrant ;

- Stockage persistant des PDFs générés sur un volume dédié convention-storage ;
- API de téléchargement sécurisée (JWT requis, vérification du propriétaire du dossier).

Événements RabbitMQ gérés :

Événement	Producteur	Action déclenchée
CandidatureAccepted	Stagiaire.Service	Génération automatique de la convention PDF
ConventionGenerated	Convention.Service	Notification au stagiaire et à l'admin (email + in-app)

4.3 Workflow de Candidature Complet



Chapitre 5 — Release 3 : Évaluations & Attestations Officielles

5.1 Evaluation.Service (.NET 8 / QuestPDF)

Développé sur le port 5072 avec sa base dédiée `evaluation_service_db`, ce microservice gère l'intégralité du processus d'évaluation pédagogique.

Types d'évaluation supportés :

- Évaluation mi-parcours : réalisée par l'encadrant à la moitié du stage, note sur 20 avec commentaires ;
- Évaluation finale : bilan complet du stage, compétences acquises, appréciation globale ;
- Validation administrative : l'administrateur RH valide ou retourne les évaluations pour correction.

Génération des attestations :

- Déclenchée automatiquement à la validation de l'évaluation finale par l'administrateur ;
- Génération du PDF officiel d'attestation de stage avec la bibliothèque QuestPDF ;
- Document incluant : identité du stagiaire, dates du stage, département, note finale, signature institutionnelle STB ;
- Téléchargement sécurisé disponible dans l'espace stagiaire.

5.2 Journal de Bord Hebdomadaire

Le journal de bord est un outil pédagogique intégré permettant :

- Au stagiaire : saisir chaque semaine les tâches réalisées, les difficultés rencontrées et les apprentissages ;
- À l'encadrant : valider les entrées, ajouter des commentaires et suivre la progression en temps réel ;
- À l'administrateur : consulter l'historique complet du parcours.

5.3 Événements RabbitMQ du Cycle d'Évaluation

Événement	Producteur	Consommateur(s)	Action déclenchée
EvaluationSubmitted	Evaluation.Service	Notification.Service	Alerte admin (note + stagiaire en attente de valid
EvaluationValidated	Evaluation.Service	Notification.Service	Email + alerte stagiaire & encadrant (note finale c

Chapitre 6 — Release 4 : IA d'Appariement des Sujets (SharpAPI)

6.1 Problématique de l'Affectation & Objectifs de l'IA

Dans les versions antérieures, l'acceptation d'une candidature par l'administrateur se limitait à affecter un département et un encadrant, sans choisir ni attribuer de thématique précise. Le stagiaire découvrait son sujet le premier jour de son arrivée, ce qui engendrait des impréparations, des déceptions mutuelles ou des demandes de réorientation complexes.

La Release 4 a introduit un changement de paradigme majeur : relier intimement l'acceptation du dossier à l'analyse automatique des compétences par IA et à l'attribution immédiate d'un sujet adapté.

6.2 Microservice SubjectMatching.Service (.NET 8)

Développé sur le port 5074 avec sa base de données dédiée `subject_matching_db`, ce microservice gère le cycle de vie des sujets de stage et orchestre l'algorithme d'appariement.

Modèle de données :

```
[Table("internship_subjects")]
public class InternshipSubject {
    [Key] public Guid Id { get; set; } = Guid.NewGuid();
    public string Title { get; set; } = string.Empty;
    public string Department { get; set; } = string.Empty;
    public string TypeStage { get; set; } = "PFE";
    [Column(TypeName = "jsonb")] public List<string> RequiredSkills { get; set; } = new();
    [Column(TypeName = "jsonb")] public List<string> PreferredSkills { get; set; } = new();
    public SubjectDifficulty Difficulty { get; set; } = SubjectDifficulty.Intermediate;
    public SubjectStatus Status { get; set; } = SubjectStatus.Open;
}
```

Listing 6.1 — Entité InternshipSubject avec stockage des compétences sous format JSONB

6.3 Intégration de l'IA SharpAPI (Parsing NLP des CVs)

Pour transformer des documents non structurés (PDF, DOCX) en données exploitables, nous avons intégré l'API d'intelligence artificielle spécialisée SharpAPI (endpoint `RH/api/v1/hr/parse_resume`). L'IA extrait automatiquement :

Les compétences techniques et méthodologiques du candidat (ex: C#, .NET, Angular, Docker, PostgreSQL) ;

- Le niveau d'études et le diplôme en cours préparé (ex: Bac+5 Ingénieur en Informatique) ;
- Les projets académiques et expériences pratiques déjà réalisés.

Résilience & Tolérance aux Pannes : Si l'API cloud externe devient inaccessible (quota, réseau), le service active automatiquement un Graceful Fallback vers un moteur d'extraction local mock, garantissant qu'aucune candidature n'est bloquée en production. Ce comportement est contrôlé par la variable d'environnement `AI_MATCHING_PROVIDER` (sharpapi ou mock).

6.4 Algorithme Déterministe de Scoring & Recommandation

Une fois le profil IA extrait, le moteur déterministe calcule un score de compatibilité normalisé entre 0% et 100% pour chaque sujet de stage ouvert :

Score = (SkillsRequises Matchées / Total Skills Requises) × 60%
+ (Skills Souhaitées Matchées / Total Skills Souhaitées) × 25%
+ Adéquation Département × 15%

Le système génère également une explication intelligible pour le responsable RH :

« Le candidat maîtrise 5/5 compétences requises (C#, .NET, PostgreSQL, Angular, Docker). 3/4 compétences souhaitées validées. Atouts identifiés : C#, .NET, Clean Architecture. Formation académique compatible avec les objectifs du stage. »

6.5 Parcours Utilisateur & Interface Interactive

Côté Administrateur RH : Lors du clic sur « Accepter », la fenêtre modale affiche immédiatement le score IA, met en évidence le sujet le plus pertinent (ex. 95%), pré-remplit la sélection et impose le choix d'un encadrant habilité.

Côté Stagiaire (« Mon Sujet ») : Dès confirmation, le stagiaire reçoit une notification et découvre sur son espace la fiche complète de son projet. S'il souhaite une autre thématique, un formulaire lui permet de formuler une demande de changement motivée, réexaminée par la direction.

6.6 Événements Asynchrones RabbitMQ du Cycle Sujet

Événement	Producteur	Consommateur(s)	Action déclenchée
SubjectProposed	SubjectMatching.Service	Notification.Service	Email officiel au candidat avec les détails du sujet
SubjectAccepted	SubjectMatching.Service	Notification.Service	Notification de confirmation à l'élève et alerte imm
SubjectChangeRequested	SubjectMatching.Service	Notification.Service	Alerte haute priorité dans le panneau admin avec
SubjectChangeReviewed	SubjectMatching.Service	Notification.Service	Notification du résultat au stagiaire

Chapitre 7 — Système de Notifications Unifié (SignalR + Email)

7.1 Architecture du Notification.Service (.NET 8)

Le Notification.Service (port 5073) joue le rôle de hub central pour toutes les communications vers les utilisateurs. Il implémente deux canaux complémentaires :

1. Notifications temps réel via SignalR : connexion WebSocket persistante entre le frontend Angular et le hub SignalR (/hub/notifications). Chaque utilisateur authentifié reçoit ses alertes instantanément dès que l'événement métier se produit.
2. Emails transactionnels SMTP : templates HTML institutionnels aux couleurs de la STB, envoyés via MailHog en développement (port 1025/8025) ou via Gmail SMTP en production.

7.2 Matrice Logique des Notifications par Rôle

Pour éviter la surcharge d'informations et garantir que chaque intervenant reçoive uniquement ce qui le concerne, une politique d'isolation stricte des notifications (ApplyReadScope) a été mise en place :

Événement	Déclencheur	Destinataire	Contenu de la Notification	Canal
Dépôt candidature	Admin RH		Nouvelle candidature déposée	In-App + Toast SignalR
Dépôt candidature	Stagiaire		Votre candidature a été enregistrée	In-App + Toast SignalR
Acceptation & Sujet	Stagiaire		Félicitations, candidature acceptée et sujet attribué	In-App + SignalR + Email
Acceptation & Sujet	Encadrant		Nouveau stagiaire attribué	In-App + Toast SignalR
Sujet validé	Admin RH		Le stagiaire a accepté le sujet	In-App + Toast SignalR
Demande de changement	Admin RH		Demande de changement de sujet	In-App + Toast SignalR
Convention prête	Stagiaire & Admin		Votre convention de stage est prête	In-App + SignalR + Email

Évaluation soumise	Admin RH	Évaluation de {Stagiaire} (In-App + SignalR + Email)
Évaluation validée	Stagiaire & Encadrant	Votre évaluation est validée In-App + SignalR + Email
Mot de passe oublié	Utilisateur	Lien de réinitialisation (valable Email uniquement)

Tableau 7.1 — Matrice de distribution des notifications par profil utilisateur

7.3 Centre de Notifications Slide-Over (Angular)

Côté frontend, un composant réactif NotificationCenterComponent a été développé. Il :

- Glisse depuis la droite de l'écran à l'activation de l'icône de cloche ;
- Propose un filtrage « Toutes » ou « Non lues » ;
- Calcule dynamiquement le badge non lu en temps réel via SignalR ;
- Permet l'action rapide « Tout marquer comme lu » ;
- Implémente le Scoping par rôle : chaque utilisateur ne voit strictement que ses propres notifications.

Chapitre 8 — Sécurité & Réinitialisation de Mot de Passe

8.1 Architecture de Sécurité

La plateforme implémente une stratégie de sécurité en profondeur (Defense in Depth) adaptée aux exigences du secteur bancaire :

- Authentification Stateless JWT : Tokens signés en HMAC-SHA256 (JWT_SECRET partagé de 48 octets minimum) avec durée de vie courte (ex: 15 minutes).
- Refresh Tokens Rotatifs : Stockés de manière sécurisée en base MySQL, renouvelés automatiquement via l'intercepteur Angular. À chaque renouvellement, l'ancien token est invalidé.
- Hachage BCrypt : Force 10 pour toutes les données sensibles (mots de passe).
- Isolation des données : Le filtre ApplyReadScope garantit qu'un utilisateur ne peut jamais accéder aux données d'un autre utilisateur.
- HTTPS : Proxy TLS Nginx disponible via le profil Docker Compose tls (certificats auto-signés en dev).

8.2 Flux de Réinitialisation Sécurisée de Mot de Passe

Remplaçant les anciens mots de passe temporaires non sécurisés, la plateforme intègre un flux cryptographique conforme aux recommandations OWASP :

1. L'utilisateur clique sur « Mot de passe oublié ? » et saisit son adresse e-mail ;
2. Le service Auth génère un token de réinitialisation unique à durée limitée (15 minutes) et le publie via RabbitMQ (PasswordResetRequested) ;
3. Le Notification.Service consomme l'événement et envoie un e-mail avec un lien direct vers la page de réinitialisation (/auth/reset-password?token=...) ;
4. L'utilisateur définit son nouveau mot de passe directement, sans qu'aucun mot de passe temporaire en clair ne soit généré ou transmis.

8.3 Journal d'Audit

Toutes les actions administratives sensibles sont journalisées avec :

- Horodatage UTC précis ;
- Identité de l'acteur (userId, rôle) ;
- Action effectuée (creation, modification, validation, refus) ;
- Entité concernée (type + identifiant) ;
- Adresse IP source.

Chapitre 9 — DevOps : Conteneurisation & Déploiement

9.1 Architecture Docker Compose (Environnement Local)

L'ensemble de la plateforme est entièrement conteneurisée avec Docker Compose. Le fichier docker-compose.yml orchestre 13 services répartis en 4 couches :

Couche Infrastructure

Conteneur	Image	Ports	Rôle
smartek-rabbitmq	rabbitmq:3-management	5672, 15672	Bus de messages asynchrones
smartek-mysql	mysql:8.0	3306	Base Auth Service
smartek-postgres	postgres:15	5432	Bases des 5 microservices .NET

smartek-mailhog	mailhog/mailhog	1025, 8025	Serveur SMTP de dev + interface web
-----------------	-----------------	------------	-------------------------------------

Couche Java / Spring

Conteneur	Port	Dépendances
smartek-eureka	8761	—
smartek-config	8888	Eureka
smartek-auth	8081	Eureka, MySQL, RabbitMQ
smartek-gateway	18080	Eureka, Config

Couche Métier .NET 8

Conteneur	Port Hôte	Port Conteneur	Base PostgreSQL
smartek-stagiaire	5070	8080	stagiaire_service_db
smartek-convention	5071	8080	convention_service_db
smartek-evaluation	5072	8080	evaluation_service_db
smartek-notification	5073	8080	notification_service_db
smartek-subject-matching	5074	8080	subject_matching_db

Couche Frontend & Proxy

Conteneur	Port	Description
smartek-frontend	4200	Angular SPA servie par Nginx
smartek-tls (opt-in)	80, 443	Proxy TLS Nginx (profil tls)

9.2 Health Checks & Dépendances de Démarrage

Chaque service définit des health checks Docker pour garantir un ordre de démarrage correct :

- Spring Boot : `curl -f http://localhost:{port}/actuator/health`
- .NET Services : `curl -fsS http://localhost:8080/health/live`
- PostgreSQL : `pg_isready -U postgres`
- MySQL : `mysqladmin ping -h localhost`
- RabbitMQ : `rabbitmq-diagnostics check_port_connectivity`

La directive `depends_on` avec `condition: service_healthy` garantit qu'un service ne

démarre que lorsque ses dépendances sont opérationnelles.

9.3 Gestion des Volumes Persistants

Deux volumes Docker nommés assurent la persistance des fichiers entre les redémarrages et reconstructions :

```
volumes:
  stagiaire-storage:  # CVs et lettres de motivation uploadés
  convention-storage: # Conventions PDF générées
```

9.4 Configuration via Variables d'Environnement

Toute la configuration sensible est externalisée dans un fichier `.env` (copié depuis `.env.example`) :

Variable	Description
JWT_SECRET	Clé HMAC-SHA256 de 48 octets minimum (obligatoire)
SHARP_API_KEY	Clé API SharpAPI pour l'analyse IA des CVs
AI_MATCHING_PROVIDER	sharpapi ou mock (fallback hors-ligne)
SMTP_HOST / SMTP_PORT	Configuration SMTP (MailHog en dev, Gmail en prod)
ADMIN_EMAIL / ADMIN_PASSWORD	Compte administrateur initialisé au premier démarrage

9.5 Proxy TLS Nginx (Profil Optionnel)

Pour les environnements de démonstration ou de pré-production, un proxy TLS Nginx est disponible :

```
bash scripts/generate-dev-certs.sh
docker compose --profile tls up -d
```

Ce proxy :

- Termine le TLS sur le port 443 et redirige HTTP (80) → HTTPS ;
- Route `/api/` et `/hub/` (SignalR WebSocket) vers la Gateway ;
- Route tout le reste vers le frontend Angular.

Chapitre 10 — Observabilité : Prometheus & Grafana

10.1 Introduction à l'Observabilité

Dans une architecture microservices, il est crucial de pouvoir observer en temps réel l'état de santé de chaque composant. La plateforme STB StageConnect intègre une stack d'observabilité complète basée sur Prometheus et Grafana.

10.2 Prometheus — Collecte des Métriques

Prometheus est un système de monitoring open-source qui collecte des métriques en interrogeant périodiquement les endpoints exposés par chaque service (scraping).

Configuration (monitoring/prometheus.yml) :

```
global:
  scrape_interval: 15s      # collecte toutes les 15 secondes
  evaluation_interval: 15s

scrape_configs:
  # Services Java Spring Boot
  - job_name: 'spring-boot-services'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets:
          - eureka-server:8761
          - auth-service:8081
          - config-server:8888
          - api-gateway:18080

  # Services .NET 8
  - job_name: 'dotnet-services'
    metrics_path: '/metrics'
    static_configs:
      - targets:
          - stagiaire-service:8080
          - convention-service:8080
          - evaluation-service:8080
          - notification-service:8080
```

```
# RabbitMQ (plugin rabbitmq_prometheus)
```

```
- job_name: 'rabbitmq'
```

```
static_configs:
```

```
- targets:
```

```
- rabbitmq:15692
```

Métriques exposées par les services :

- Services Spring Boot : via Spring Boot Actuator + endpoint /actuator/prometheus — métriques JVM (heap, GC), HTTP request rates, thread pools ;
- Services .NET 8 : via Smartek.Common → UseSmartekMetrics() (prometheus-net) — requêtes HTTP par code de statut, durée des requêtes, taille des payloads, connexions base de données ;
- RabbitMQ : via le plugin rabbitmq_prometheus sur le port 15692 — profondeur des queues, messages publiés/consommés/en attente.

10.3 Grafana — Visualisation des Métriques

Grafana est un outil de visualisation qui connecte Prometheus comme source de données et propose des tableaux de bord interactifs.

Dashboard disponible (monitoring/grafana-dashboard.json) :

- Vue globale de la santé de tous les microservices ;
- Taux de requêtes HTTP et codes d'erreur par service ;
- Latence des requêtes (P50, P95, P99) ;
- Métriques JVM pour les services Java ;
- État des queues RabbitMQ en temps réel ;
- Alertes configurables par seuil (ex: taux d'erreur > 5%).

10.4 Instrumentation des Services .NET 8

La bibliothèque partagée Smartek.Common expose une extension UseSmartekMetrics() appelée dans chaque Program.cs des microservices .NET :

```
// Dans Program.cs de chaque microservice .NET
```

```
app.UseSmartekMetrics(); // Expose /metrics pour Prometheus
```

```
app.UseSmartekHealthChecks(); // Expose /health/live et /health/ready
```

Cette approche garantit une instrumentation uniforme et centralisée sans duplication de code.

10.5 Rôle dans le Contexte Bancaire

Dans le secteur bancaire, la disponibilité et la performance des systèmes sont des exigences critiques (SLA). L'observabilité via Prometheus/Grafana permet :

- Détection proactive des anomalies avant qu'elles n'affectent les utilisateurs ;
- Traçabilité des incidents pour les audits réglementaires ;
- Capacity planning basé sur les métriques historiques réelles.

Chapitre 11 — Orchestration Kubernetes

11.1 Introduction à Kubernetes

Kubernetes (K8s) est la plateforme d'orchestration de conteneurs de référence dans l'industrie. Dans le projet STB StageConnect, les manifestes Kubernetes présents dans le répertoire `k8s/` permettent de déployer l'ensemble de la plateforme sur un cluster K8s (Minikube/Docker Desktop en dev, cluster managé en production).

11.2 Structure des Manifestes K8s

k8s/	
├─ namespace.yaml	# Namespace dédié : gestion-stagiaires-stb
├─ configmap.yaml	# Configuration non-sensible (URLs, paramètres)
├─ secret.yaml	# Secrets chiffrés (JWT_SECRET, mots de passe DB)
├─ deployment.yaml	# Déploiements de tous les microservices
├─ service.yaml	# Services Kubernetes (ClusterIP / LoadBalancer)
├─ ingress.yaml	# Exposition externe via Nginx Ingress Controller
└─ hpa.yaml	# HorizontalPodAutoscalers (autoscaling)

11.3 Namespace Dédié

Tous les composants sont isolés dans un namespace Kubernetes dédié :

apiVersion: v1
kind: Namespace
metadata:
name: gestion-stagiaires-stb

11.4 Autoscaling Horizontal (HPA)

L'un des avantages clés de Kubernetes est sa capacité à ajuster automatiquement le nombre de pods en fonction de la charge. Le fichier `hpa.yaml` définit des `HorizontalPodAutoscalers` pour les services critiques :

Service	Min Pods	Max Pods	Seuil CPU	Seuil Mémoire
API Gateway	3	9	70%	80%
Stagiaire.Service	3	8	70%	—
Convention.Service	2	6	75%	—
Evaluation.Service	2	6	75%	—
Notification.Service	2	6	75%	—
Frontend	2	6	75%	—

Tableau 11.1 — Politique d'autoscaling par service

Comportement de scale-down maîtrisé : Une fenêtre de stabilisation de 300 secondes est configurée pour éviter le flapping (montées/descentes en cascade) lors de pics de charge brefs.

11.5 Ingress & Exposition Externe

Le fichier `ingress.yaml` configure un Nginx Ingress Controller pour exposer les services via des routes HTTP/HTTPS :

Routes configurées :
/api/v1/* → API Gateway (port 18080)
/hub/* → Notification.Service SignalR (WebSocket upgrade)
/* → Frontend Angular

11.6 Avantages Kubernetes vs Docker Compose

Critère	Docker Compose	Kubernetes
Environnement cible	Développement local	Production / Staging
Autoscaling	Manuel	Automatique (HPA)
Self-healing	Limité	Restart automatique des pods défaillants

Load balancing	Basique	Natif + Ingress Controller
Rolling updates		Zero-downtime deployments
Secrets management	Fichier .env	Kubernetes Secrets chiffrés

Chapitre 12 — Tests & Qualité Logicielle

12.1 Stratégie de Tests

Une couverture de tests rigoureuse a été mise en place pour garantir la fiabilité de la plateforme :

Projet de tests : Stagiaire.Service.Tests

Stagiaire.Service.Tests/	
├─ Unit/	# Tests unitaires des services et validateurs
├─ Integration/	# Tests d'intégration avec base de données réelle (PostgreSQL)
└─ Fixtures/	# Données de test partagées

12.2 Tests Unitaires

Les tests unitaires vérifient la logique métier isolément (sans base de données ni dépendances externes) :

- Moteur de scoring IA : validation des formules d'appariement pour chaque combinaison de compétences ;
- Validateurs de domaine : règles de gestion (ex: une candidature ne peut pas être acceptée deux fois) ;
- Transformations DTO : conversion entre entités et objets de transfert.

12.3 Tests d'Intégration

Les tests d'intégration utilisent une instance PostgreSQL réelle (via Testcontainers ou base de données dédiée aux tests) pour valider :

- Les migrations EF Core et l'intégrité du schéma ;
- Les opérations CRUD sur les candidatures, évaluations et sujets ;
- Les comportements transactionnels (rollback en cas d'erreur).

12.4 Validation par Swagger UI

Chaque microservice .NET expose une interface Swagger UI permettant de tester interactivement toutes les APIs REST :

Service	URL Swagger
Stagiaire.Service	http://localhost:5070/swagger
SubjectMatching.Service	http://localhost:5074/swagger
Convention.Service	http://localhost:5071/swagger
Evaluation.Service	http://localhost:5072/swagger
Notification.Service	http://localhost:5073/swagger

12.5 Tests Manuels & Comptes de Démonstration

Un jeu de données de démonstration complet permet de valider le scénario bout-en-bout :

Rôle	Adresse E-mail	Mot de Passe	Description
ADMIN RH	admin@stb.tn	Admin123!	Compte administrateur avec tous les droits
STAGIAIRE	gadourkaddachi000@gma	Stagiaire123!	Compte stagiaire avec candidature et sujet propos
ENCADRANT	encadrant@stb.tn	Encadrant123!	Compte encadrant créé via le panneau admin

Chapitre 13 — Difficultés Techniques & Résolutions

13.1 Conversion d'Énumérations sous EF Core

Problème : L'entité InternshipSubject configurée avec HasConversion<string>() rejetait les filtres SQL lorsque les données en base contenaient l'entier brut au lieu des chaînes de caractères attendues.

Solution : Normalisation des données existantes vers les valeurs textuelles ('Open', 'Closed') et ajout d'une tolérance logicielle dans les requêtes LINQ pour accepter les deux formats.

13.2 Règles de Réactivité Angular Signal Writes

Problème : L'exécution d'effets réactifs Angular déclenchant des écritures synchrones levait l'exception NG0600 (signal write in reactive context).

Solution : Application de `untracked()` pour les lectures hors contexte réactif et de l'option `{ allowSignalWrites: true }` sur les effets nécessitant des mises à jour synchrones. Cela a restauré la fluidité du composant de notification.

13.3 Scoping & Étanchéité des Données

Problème : L'administration recevait des notifications personnelles destinées aux candidats, et inversement.

Solution : Implémentation du filtre dynamique `ApplyReadScope` dans le `Notification.Service`, garantissant que chaque rôle ne consulte que son périmètre légitime de notifications.

13.4 Health Checks & Ordre de Démarrage Docker

Problème : Des conditions de course (race conditions) faisaient démarrer des microservices avant que leurs dépendances (PostgreSQL, RabbitMQ) soient pleinement opérationnelles, causant des erreurs de connexion au démarrage.

Solution : Configuration systématique des health checks Docker avec `depends_on: condition: service_healthy` et `start_period: 45s` pour les microservices .NET (qui appliquent leurs migrations EF Core au premier démarrage).

13.5 Gestion du JWT_SECRET Partagé

Problème : Dans une architecture hybride Java/.NET, le secret JWT doit être partagé entre l'Auth Service (Java, émetteur) et les microservices .NET (consommateurs/validateurs).

Solution : Standardisation de la lecture de la variable d'environnement `JWT_SECRET` dans tous les services, avec validation obligatoire au démarrage (erreur explicite si absente).

13.6 SignalR & WebSocket derrière Nginx

Problème : Les connexions SignalR WebSocket étaient interrompues par le proxy Nginx qui ne transmettait pas les en-têtes `Upgrade` et `Connection` nécessaires.

Solution : Configuration explicite du proxy Nginx pour transmettre les en-têtes WebSocket :

```
proxy_set_header Upgrade $http_upgrade;
```

```
proxy_set_header Connection "upgrade";
```

Conclusion Générale & Bilan

Bilan Fonctionnel

Ce stage d'été au sein de la Société Tunisienne de Banque a permis de concevoir et de déployer une solution logicielle d'envergure industrielle. La plateforme STB StageConnect digitalise entièrement le cycle de vie du stagiaire, de l'inscription à l'attestation de stage, en passant par l'appariement intelligent par IA, la génération automatique de documents PDF, les évaluations pédagogiques et le système de notifications temps réel.

Bilan Technique

Axe	Réalisations
Architecture	9 microservices interconnectés, pattern Database-per-Se
Backend Java	Spring Cloud Gateway, Auth JWT, Eureka, Config Server
Backend .NET 8	5 microservices spécialisés, EF Core, QuestPDF, SignalR
IA	Intégration SharpAPI NLP, moteur de scoring déterminist
Frontend	Angular 18+ avec signaux réactifs, TailwindCSS, notific
DevOps	Docker Compose (13 services), Kubernetes (HPA, Ingress,
Observabilité	Prometheus (scraping 15s), Grafana (dashboards), health
Sécurité	JWT HMAC-SHA256, refresh tokens rotatifs, BCrypt, reset
Tests	Tests unitaires & intégration (Stagiaire.Service.Tests)

Compétences Acquises

Ce projet a permis de consolider et d'acquérir les compétences suivantes :

- Conception d'architectures microservices hybrides (Java Spring Cloud + .NET 8) ;
- Développement backend avancé : EF Core, MassTransit, SignalR, QuestPDF ;
- Intégration d'APIs d'Intelligence Artificielle (NLP, parsing sémantique) ;
-

Développement frontend réactif avec Angular 18 (signaux, intercepteurs, composants autonomes) ;

- DevOps & Infrastructure : Docker Compose multi-services, Kubernetes, Nginx, CI/CD ;
 - Observabilité : Prometheus, Grafana, métriques système et métier ;
 - Sécurité applicative : JWT, hachage, flux OWASP, isolation des données.
-

Remerciements

J'adresse mes plus vifs remerciements à mon maître de stage, M. Makrem TAIEB, pour sa confiance, son expertise technique et ses précieux conseils tout au long de cette mission. Mes remerciements s'étendent à toute l'équipe de la STB pour leur accueil bienveillant, ainsi qu'au corps professoral de l'ESPRIT pour l'excellence de la formation d'ingénieur dispensée.

Rapport rédigé et soutenu par Abdelkader KADDACHI — Diplôme National d'Ingénieur en Informatique — ESPRIT 2026.